TEACH YOURSELF INTERMEDIATE XI UNIT 2

Computational Thinking & Algorithms

Problem decomposition, patterns, and classic sort/search

What you will master

- Apply decomposition, pattern recognition, and abstraction
- Write algorithms and pseudocode for real problems
- Implement and trace bubble sort and selection sort (any two sorts)
- Apply linear and binary search (any two searches)

How to use these notes

- Read one section, then close the book and explain it aloud.
- Do every *Try it* box before moving on.
- Memorize tables and truth patterns — exams love them.

2.1 What is computational thinking?

Computational thinking is a disciplined way to solve problems: break them down, find patterns, hide irrelevant detail, and design step-by-step procedures (**algorithms**).

An **algorithm** is language-independent logic. **Pseudocode** is human-readable structure (IF, FOR, WHILE) that programmers translate into code.

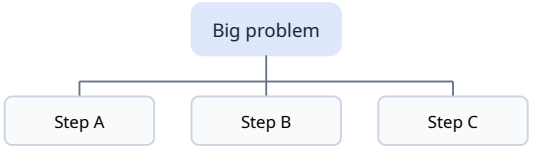


Figure 2.1 — Decomposition turns one hard task into manageable subtasks.

2.2 Algorithmic strategies

Decomposition

Split complex jobs (e.g. learning a language: vocabulary, grammar, tenses, practice).

Pattern recognition

Spot repetition — row has stars → use a loop with counter.

Abstraction

Keep essential steps (boil water, steep tea); ignore cup color or kettle brand.

Good algorithms are **clear**, **correct**, **efficient** in time/memory, and **reusable**.

2.3 Bubble sort

Bubble sort compares adjacent pairs and swaps if out of order; largest values “bubble” to the end each pass.

For items, worst-case passes relate to comparisons — fine for small lists, slow for huge data.

```
FOR outer = 0 TO n-2
  FOR inner = 0 TO n-2-outer
    IF list[inner] > list[inner+1] THEN swap
  END FOR
END FOR
```

> **Try it**
> Sort {8,4,1,9,3} by hand for two full passes; write swaps after each comparison.

2.4 Selection sort & searching

Selection sort finds the minimum in the unsorted portion and swaps it into place — also but fewer swaps than bubble sort.

Linear search scans until found or end — works on unsorted data, .

Binary search requires a *sorted* list; halve the search range each step — .

> **Exam tip**
> State preconditions: binary search needs sorted input; bubble/selection do not.

Teach Yourself — Practice & Check

1. Write pseudocode to find the largest of three numbers.
2. Trace bubble sort on ${C`{5,2,8,1}`} until sorted.
3. How many comparisons for binary search on 16 sorted items (worst case)?
4. Give one real-life use of pattern recognition in apps you use daily.